

Career Fair Eligibility Application

Tech Stack: HTML , CSS, Vanilla CSS

Github- <https://github.com/Chetan175/Cisco-Project.git>

Step 0- Understanding the Requirements-

1) Create the Student Profile Form

- a) Create a form where the student can enter/edit:
 - i) Branch
 - ii) CGPA
 - iii) Graduation Year
 - iv) Active Backlogs
 - v) Skills
- b) Provide an Evaluate button.
- c) Provide Sample and Reset controls.
- d) Load the given student profile as the default/sample data:
 - i) Branch: CSE
 - ii) CGPA: 8.1
 - iii) Graduation Year: 2027
 - iv) Active Backlogs: 1
 - v) Skills: Git, Python, SQL

2) Enforce Input Constraints

- a) Branch cannot be blank.
- b) CGPA must be a finite number from 0 to 10, inclusive.
- c) Graduation year must be a whole number from 2000 to 2100.
- d) Active backlogs must be a whole number ≥ 0 .
- e) Display the appropriate validation error:
 - i) INVALID_BRANCH
 - ii) INVALID_CGPA
 - iii) INVALID_GRADUATION_YEAR
 - iv) INVALID_BACKLOG_COUNT

3) Normalize Student Input

- a) Trim spaces from the branch.
- b) Trim spaces from skills.
- c) Compare branch values case-insensitively.
- d) Compare skill values case-insensitively.
- e) Split the skill input using commas.
- f) Ignore empty skill values.
- g) Remove duplicate skills.
- h) Do not assume aliases or related skills are equivalent.

4) **Define the Fixed Role Data**

- Create the five fixed roles with their requirements:
 - **CF01 — Data Operations Intern:** Branch: CSE, IT; Min CGPA: 7.5; Grad year: 2027; Max backlogs: 1; Skills: Python, SQL
 - **CF02 — QA Automation Intern:** Branch: CSE, ECE, IT; Min CGPA: 7.0; Grad years: 2027, 2028; Max backlogs: 1; Skill: Git
 - **CF03 — Embedded Systems Intern:** Branch: ECE, EEE; Min CGPA: 7.5; Grad year: 2027; Max backlogs: 1; Skill: Git
 - **CF04 — Machine Learning Intern:** Branch: CSE, IT; Min CGPA: 8.5; Grad year: 2027; Max backlogs: 1; Skill: Python
 - **CF05 — Platform Engineering Intern:** Branch: CSE, ECE; Min CGPA: 7.0; Grad year: 2026; Max backlogs: 0; Skills: Docker, Git

5) **Implement the Eligibility Engine**

- For every role, independently check:
 - Is the student's branch allowed?
 - Is the CGPA greater than or equal to the minimum?
 - Is the graduation year allowed?
 - Are active backlogs within the maximum?
 - Does the student have every required skill?
- A role is ELIGIBLE only if all five checks pass.

6) **Collect All Failure Reasons**

- Do not stop at the first failed rule.
- Check every rule independently.
- Store every failed condition.
- Use these exact failure codes:
 - `BRANCH_NOT_ALLOWED`
 - `CGPA_BELOW_MINIMUM`
 - `GRADUATION_YEAR_NOT_ALLOWED`
 - `TOO_MANY_ACTIVE_BACKLOGS`
 - `MISSING_SKILL: <skill>`
- Maintain the required failure-reason order.
- Sort multiple missing skills alphabetically, case-insensitively.

7) **Generate the Eligibility Results**

- For every role, generate a result containing:
 - Role ID
 - Role title
 - Eligibility status
 - Failure reasons

8) **Sort the Results**

- Divide results into ELIGIBLE and INELIGIBLE.
- Show all eligible roles first.
- Show all ineligible roles afterward.
- Within each group, sort by role title alphabetically, case-insensitively.

- If titles are the same, sort by role ID.
- 9) **Display Results in the UI**
 - Create a results section showing:
 - Eligible roles
 - Ineligible roles
 - Failure reasons for ineligible roles
 - Eligible count
 - Ineligible count

Step 1- Implementation Plan-

Implementation Plan Prompt -

I am providing you a PDF named Probelm_Statement containing the complete requirements for my project. Read and understand it carefully before doing anything. Act as a senior software engineer and create a complete, practical implementation plan for this project using HTML, CSS, and JavaScript. Cover:

1. Project understanding — features, pages, user flow, and expected behavior.
2. Implementation plan — what to build, in what order, and how the HTML, CSS, and JS should work together.
3. All possible scenarios — normal cases, invalid/empty inputs, edge cases, unusual user actions, refreshes, multiple clicks, DOM/JS errors, responsive/mobile issues, and project-specific cases.
4. Testing — give concrete test cases as Action/Input → Expected Result and try to deliberately break the website.
5. Final checklist — verify that every requirement and feature from the PDF has been implemented and tested.

Keep everything simple, practical, and specific to this project. Do not give generic advice. If anything in the PDF is unclear or missing, clearly point it out instead of making assumptions. Do not start coding immediately. First understand → plan → identify edge cases → define tests → then implement

Comparing and Evaluating the Requirements of the Problem Statement we proceed with implementation plan given by Claude.

Moving to Gemini Pro model for implementation

Step 2-(Giving the exact guidelines for antigravity on which, it will function, and setting up the modular file structure)

Prompt - 1: (From the implementation plan created from Claude, we tell the instructions to Model on how to operate)

I've shared the problem statement and implementation plan. Read both carefully and completely before we start. Before you touch any code, confirm your understanding back to me in 3-4 bullets so I can catch any misreadings early.

Ground rules for this whole session: - We'll build step by step. I will give you prompts one at a time, only do what I explicitly ask, don't assume or add anything on your own.

If anything is unclear, ask me before writing or changing code. DO NOT GUESS.

Tech stack: Vanilla HTML, CSS, and JavaScript. No frameworks, no build tools.

Understood. I have thoroughly reviewed the problem statement and the implementation plan. Here is my understanding of the core requirements and constraints:

- **Core Goal & Tech Stack:** Build a client-side, single-page application using only Vanilla HTML, CSS, and JavaScript (no frameworks or build tools). It will evaluate a single student's profile against 5 fixed, hardcoded roles to determine eligibility.
- **Strict Evaluation & Ordering Rules:** The logic must evaluate 5 fields (Branch, CGPA, Grad Year, Backlogs, Skills). Results must be strictly ordered (Eligible first, then alphabetically by role title). Failure reasons for ineligible roles must appear in an exact, predefined sequence (Branch, then CGPA, then Grad Year, then Backlogs, then alphabetical Missing Skills).
- **Validation & State Management:** The UI must display specific error codes (`INVALID_BRANCH`, etc.) for invalid inputs. Any invalid input must immediately clear all role results and counts so no stale data is shown.
- **Working Agreement:** I will strictly follow your step-by-step instructions, implement *only* what you explicitly ask for without making assumptions or adding extra features, and I will ask for clarification if anything is ambiguous rather than guessing.

Prompt - 2: Creation of clean, simple file structure

Before any logical code, create a basic skeleton file structure with empty files:
data.js, validate.js, engine.js, ui.js, index.html, style.css, test_validate.js, test_engine.js.

In data.js, export exactly this built-in sample data as :

```
const students = [
  {
    id: "STU01",
    name: "Student 1",
    branch: "CSE",
    cgpa: 8.1,
    graduationYear: 2027,
    activeBacklogs: 1,
    skills: ["Git", "Python", "SQL"]
  }, {
    id: "STU02",
    name: "Student 2",
    branch: "ECE",
    cgpa: 7.5,
    graduationYear: 2027,
    activeBacklogs: 0,
    skills: ["Git"]
  }
];
```

The above data only serves as the hardcoded sample data that showcases the format of details , we always fetch the data from HTML input

Prompt 3- Implement only the student profile form in index.html.

Create a Student Profile section with these input fields:

Do not implement validation logic yet

1. Branch

- Use a dropdown/select.
- Options:
 - CSE
 - IT
 - ECE
 - EEE

2. CGPA

- Use a text/number input field.

3. Graduation Year

- Use a text/number input field.

4. Active Backlogs

- Use a text/number input field.

5. Skills

- Use a text input field.
- The user will enter comma-separated skills, for example:
Git, Python, SQL

Also add:

- Evaluate button
- Reset button

For now, these buttons do not need functionality.

Only create the HTML form structure with clear labels, ids, and names for each field.

Prompt 4-Implement the form → validate.js flow.

When Evaluate is clicked: - Collect branch, CGPA, graduation year, active backlogs, and skills from the form. - Create a student object and pass it to validate.js. - Let validate.js perform all required validation. - Display the returned validation message/result on the page. Keep validation logic only in validate.js. Use the existing HTML constraints as the first layer, but ensure JavaScript validation follows the problem statement.

Prompt 5- Normalize Validated Input

Now implement input normalization in validate.js. Before validation/evaluation: - Trim whitespace from branch and skills. - Split skills by commas. - Ignore empty skills. - Remove duplicate skills case-insensitively. - Make skills suitable for case-insensitive comparison. Return the normalized student data along with the validation result. Do not implement eligibility logic or modify the UI yet.

Prompt 6- Eligibility Logic

Implement only the eligibility logic in engine.js.

For each role, check:

- Branch is allowed
- CGPA $\geq$ minimum CGPA
- Graduation year is allowed
- Active backlogs $\leq$ maximum
- Every required skill is present

Return each role as ELIGIBLE or INELIGIBLE with all failed rules.

Use the exact failure codes and order from the problem statement.

Do not modify the HTML, CSS, validation.js, or UI logic yet.

Do not render results yet.

Prompt 7- Connect Eligibility Engine to the UI

Now connect the existing validate.js and engine.js to ui.js.

When Evaluate is clicked:

- Collect the form data.
- Validate and normalize it.
- If invalid, show the validation error and stop.
- If valid, pass the student data to engine.js.
- Get the eligibility results from the engine.
- Display the results on the page.
- Show eligible and ineligible counts.

Do not change the eligibility logic or validation logic.

Do not add new rules.

Prompt 8- Display Final Results on UI

Now complete the results handling in ui.js.

After getting the eligibility results:

- Show ELIGIBLE roles first.
- Show INELIGIBLE roles second.
- Within each group, sort by role title case-insensitively, then role ID.
- Display the eligible count and ineligible count.
- Display every failure reason in the order returned by engine.js.
- Clear old results and counts before each new evaluation.
- If validation fails, clear all previous results and counts.

Do not change validate.js or engine.js.

Do not add new eligibility rules.

Testing out the Project Flow using Live Server and using console if there is a data flow issue-

< > ↺ 🔖 🔔 127.0.0.1:5500/index.html

Career Fair Eligibility Shortlist

Student Profile

Branch

CGPA

Graduation Year

Active Backlogs

Skills

Eligible: 1 | Ineligible: 4

QA Automation Intern (CF02) - ELIGIBLE

Data Operations Intern (CF01) - INELIGIBLE

- GRADUATION_YEAR_NOT_ALLOWED
- MISSING_SKILL: Python
- MISSING_SKILL: SQL

Embedded Systems Intern (CF03) - INELIGIBLE

- BRANCH_NOT_ALLOWED
- GRADUATION_YEAR_NOT_ALLOWED

Prompt 9- Improving CSS

Now make the final UI/CSS improvements in style.css.

- Use Flexbox for the main layout and form sections.
- Center-align the main content and form elements.
- Add consistent gaps between all input fields.
- Improve spacing, typography, buttons, cards, and overall visual hierarchy.
- Eligible role cards/results should have a light green background.
- Ineligible role cards/results should have a light red background.
- Keep the design simple and professional.
- Do not change any JavaScript, validation, eligibility, or data logic.
- Do not add new functionality.
- Ensure the existing functionality continues to work after the CSS changes.

INPUT FORM-

Career Fair Eligibility Shortlist

Student Profile

Branch

CSE

CGPA

8.5

Graduation Year

2028

Active Backlogs

0

Skills

Git, Python , SQL

Evaluate

Reset

OUTPUT -

Eligible: 1 | Ineligible: 4

QA Automation Intern (CF02) - ELIGIBLE

Data Operations Intern (CF01) - INELIGIBLE

- GRADUATION_YEAR_NOT_ALLOWED

Embedded Systems Intern (CF03) - INELIGIBLE

- BRANCH_NOT_ALLOWED
- GRADUATION_YEAR_NOT_ALLOWED

Machine Learning Intern (CF04) - INELIGIBLE

- GRADUATION_YEAR_NOT_ALLOWED

Platform Engineering Intern (CF05) - INELIGIBLE

- GRADUATION_YEAR_NOT_ALLOWED
- MISSING_SKILL: Docker

After some Testing found out that the reset button implementation was missing.

Prompt 10-Reset Button Functionality

Implement the reset functionality in `ui.js`:

1. Select `#reset-btn` and `#profile-form` using `getElementById()`.
2. Attach a `click` event listener to the reset button.
3. Inside the event handler:
 - Call `profileForm.reset()`.
 - Clear `validationMessageEl.textContent`.
 - Reset the inline color style of `validationMessageEl`.
 - Clear `countsContainer.textContent`.
 - Remove all generated result elements using `resultsContainer.innerHTML = ''`.